

Labelle.tn — E-commerce Data Architecture

New entities, their relationships, and how the existing inventory schema is reused

Prepared for the Labelle project · June 12, 2026 · Backend: Symfony 5.4 / Doctrine ORM / MySQL · Storefront: Angular (labelle.tn)
· Back-office: labelle-inventory-ui

1. The four questions, answered up front

Q1 — Category and sub-category: one entity or two?

ONE ENTITY A single `store_category` table with a nullable `parent_id` pointing to itself (an "adjacency list"). Your menu is three levels deep (Visage → Peaux → ...) and a self-referencing tree handles any depth with one table, one Doctrine entity, and one set of CRUD screens. Two separate tables would hard-code the depth and break the day the client wants a fourth level or wants to move a sub-category up.

Precedent: PrestaShop (`ps_category.id_parent`), Sylius (`Taxon` tree), WooCommerce (term hierarchy), Magento — every major platform models categories as one self-referencing entity. None uses a separate sub-category table.

Q2 — Product details table joined to the existing product table?

YES — ONE-TO-ONE, OPT-IN A `store_product_detail` table with a **unique** FK to `product`. The inventory `product` table stays the single source of truth for stock and prices; the detail row adds everything the web needs (descriptions, images, slug, categories, publish flag). Crucially, a product is only on the website **if a detail row exists and is published** — the warehouse can hold 2,410 SKUs while the shop sells a curated 300. Inventory screens and reports are completely unaffected.

Q3 — A separate `store_user` table for shoppers?

YES — AND LINK IT TO CLIENT Staff and shoppers must never share a credentials table: different auth firewalls, different lifecycle, different risk profile (a leaked shopper password must not be a staff password). One refinement to your idea: keep the existing `client` table as the **customer profile** (it already holds every buyer across all channels, keyed by phone), and make `store_user` the **login account** that links to a client. A shopper who already bought in the boutique and then registers online gets connected to their full purchase history by phone number.

Precedent: Sylius — the reference Symfony e-commerce framework — splits exactly this way: `Customer` (profile, may exist without login) vs `ShopUser` (credentials) vs `AdminUser` (staff). PrestaShop likewise separates `ps_customer` from `ps_employee`. Your `client` / `store_user` / `user` trio maps onto this one-to-one.

Q4 — Web orders: reuse the existing `t_order` table?

YES — EXTEND, DON'T DUPLICATE Reuse `t_order` as the single financial and stock record, extended two ways: a new `source` column (`manual` | `web` | `intigo`) and a one-to-one `store_order_detail` row carrying web-only data (status, shipping address, fee, payment method). Reasons:

- **Stock:** `t_order` creation already decrements `product.quantity` atomically. A separate web-order table would need a duplicate of that logic — the #1 source of oversell bugs.
- **Reporting:** every KPI, chart, revenue report, and inventory snapshot reads `t_order`. Web sales appear in the client's existing dashboards on day one, for free.
- **Proven pattern:** the Intigo marketplace sync already pushes external orders into `t_order`. The web shop is just a second external channel.
- **Staff workflow:** web orders show up in the existing order list the team uses daily — filterable by source, no second screen to watch.

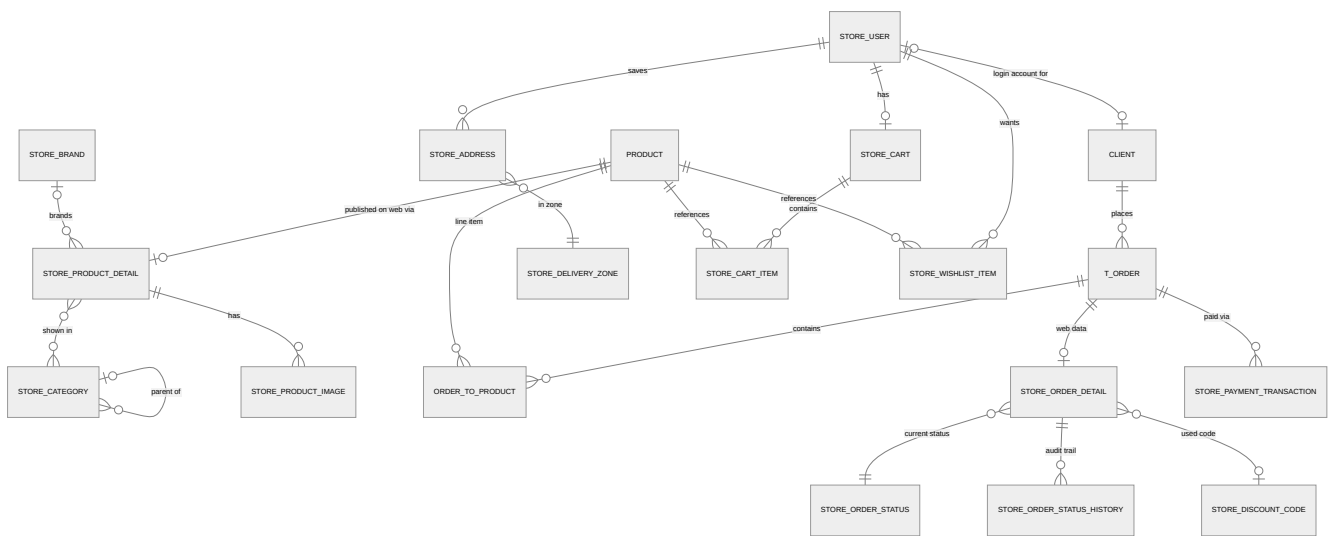
Precedent: Shopify stores every order from every channel (online store, POS, marketplaces) in one orders table with a `source_name` column. PrestaShop keeps one `ps_orders` table and puts state into a separate `ps_order_state` + `ps_order_history` pair — which is exactly the status model proposed in §5.

2. Design principles

- **One source of truth.** Stock = `product.quantity`. Sales = `t_order`. Customers = `client`. New tables *decorate*, they never duplicate.
- **Never break the inventory app.** Every change to an existing table is additive (new nullable/defaulted columns only). Existing screens, services and reports keep working unmodified.
- **Visual and namespace separation.** Every new table is prefixed `store_`. In the back-office, the e-commerce module lives in its own sidebar group; in the schema, the prefix gives the same separation.
- **Snapshot what history must not rewrite.** Orders copy the shipping address, prices, fees and discount amounts at purchase time. Editing a customer's address must never change a shipped order. (`t_order` already does this for client name/phone — the pattern is extended, not invented.)
- **Proper `utf8mb4` from day one.** All new tables are created with `utf8mb4_unicode_ci` so the legacy `Utf8Encoder` workaround is never needed on the new API surface.
- **Money columns are `DECIMAL(10,3)`.** The Tunisian dinar has 3 decimals (millimes). Legacy float columns stay as they are; new ones do it right.
- **Deliberately no EAV, no variants engine.** Magento's attribute-value model is the canonical example of flexibility nobody asked for becoming everyone's performance problem. Each sellable item here is one inventory product with its own barcode — which is how the warehouse already works. See §8 for the escape hatch if variants are ever needed.

3. The big picture

■ White = existing tables (unchanged or additive-only) ■ `store_*` = new tables



Existing tables not shown (untouched): `user` , `role` , `product_history` , `broken_product` , `document` , `folder` , `inventory_snapshot` .
 Support tables not shown: `store_setting` , `store_contact_message` , `store_email_log` , `store_newsletter_subscription` , `store_product_review` , `store_banner` .

Build phases used throughout this document: **PHASE 1** Catalog — the storefront stops using static data **PHASE 2** Checkout — guest orders, COD **PHASE 3** Accounts — login, addresses, wishlist, cart sync **PHASE 4** Growth — promo codes, online payment, reviews, banners

4. Catalog entities PHASE 1

4.1 `store_category` — one tree for categories and sub-categories

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>parent_id</code>	INT NULL, FK → <code>store_category.id</code>	NULL = root category. <code>ON DELETE RESTRICT</code> — a category with children cannot be deleted by accident.
<code>name</code>	VARCHAR(128)	"Visage", "Peaux", ...
<code>slug</code>	VARCHAR(160) UNIQUE	URL segment; the storefront already routes like <code>/3-visage</code> — slugs replace the scraped numeric links.
<code>description</code>	TEXT NULL	Optional intro text on the category page (good for SEO).
<code>image</code>	VARCHAR(255) NULL	Banner/thumbnail path.
<code>position</code>	INT DEFAULT 0	Manual ordering inside the parent.
<code>is_active</code>	TINYINT(1) DEFAULT 1	Hide a whole branch without deleting it.
<code>created_at</code> / <code>updated_at</code>	DATETIME	

The storefront menu (currently the hard-coded `MENU` array) is built by one query ordered by `parent_id`, `position` and assembled into a tree in PHP — at this catalog size (~40 categories) no nested-set optimization is warranted.

4.2 `store_product_detail` — the product's web identity

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>product_id</code>	INT UNIQUE , FK → <code>product.id</code>	One-to-one. <code>ON DELETE CASCADE</code> — deleting an inventory product removes its web presence.
<code>slug</code>	VARCHAR(180) UNIQUE	e.g. <code>creme-hydratante-50ml</code> ; replaces scraped numeric product links.
<code>display_name</code>	VARCHAR(180) NULL	Customer-facing name. NULL → falls back to <code>product.name</code> (warehouse names are often abbreviated; the web name can be prettier without touching inventory data).
<code>card_name</code>	VARCHAR(80) NULL	Short name for product cards (your <code>Product.cardName</code> field).
<code>short_description</code>	TEXT NULL	Bullet lines under the title (storefront already renders <code>shortDescription[]</code>).
<code>description</code>	LONGTEXT NULL	Full HTML description tab.
<code>brand_id</code>	INT NULL, FK → <code>store_brand.id</code>	<code>ON DELETE SET NULL</code> . Beauty shoppers filter by brand constantly — cheap now, painful to retrofit.
<code>default_category_id</code>	INT NULL, FK → <code>store_category.id</code>	The category used for the breadcrumb when a product is in several categories. <code>ON DELETE SET NULL</code> . (PrestaShop: <code>id_category_default</code> .)
<code>promo_price</code>	DECIMAL(10,3) NULL	Sale price. NULL = sells at <code>product.selling_price</code> . The web never overrides the base price — it comes from inventory; promo is an explicit, time-boxed exception.
<code>promo_starts_at</code> / <code>promo_ends_at</code>	DATETIME NULL	Both NULL = promo always on (while <code>promo_price</code> set).
<code>is_published</code>	TINYINT(1) DEFAULT 0	The master switch. Draft until a human says go.
<code>is_featured</code>	TINYINT(1) DEFAULT 0	Feeds the homepage "featured products" block.

<code>meta_title / meta_description</code>	VARCHAR(255) NULL	SEO overrides.
<code>created_at / updated_at</code>	DATETIME	

What the storefront reads per product: name/cardName/descriptions/images/slug from `store_product_detail` ; `price` from `product.selling_price` (or active `promo_price`); `inStock / stock` from `product.quantity` . Because orders decrement `product.quantity` at creation (§5.4), the displayed stock is always real — no separate "reserved stock" table is needed.

4.3 `store_product_to_category` — many-to-many junction

Column	Type	Notes
<code>product_detail_id</code>	INT, FK → <code>store_product_detail.id</code>	Composite PK with <code>category_id</code> ; <code>ON DELETE CASCADE</code> both sides.
<code>category_id</code>	INT, FK → <code>store_category.id</code>	A product can sit in "Soins visage" <i>and</i> "Promotions" <i>and</i> "Nouveautés".

4.4 `store_product_image`

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>product_detail_id</code>	INT, FK → <code>store_product_detail.id</code>	<code>ON DELETE CASCADE</code> .
<code>path</code>	VARCHAR(255)	File on disk under the backend's <code>public/uploads/products/</code> — same disk-plus-metadata pattern as the existing <code>document</code> module. Upload from the back-office; resize to card/large variants at upload time.
<code>alt</code>	VARCHAR(160) NULL	SEO/accessibility.
<code>position</code>	INT DEFAULT 0	Gallery order.
<code>is_cover</code>	TINYINT(1) DEFAULT 0	The card image. Exactly one per product (enforced in service code).

4.5 `store_brand`

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>name</code>	VARCHAR(128)	
<code>slug</code>	VARCHAR(160) UNIQUE	Brand listing page <code>/marque/avene</code> .
<code>logo</code>	VARCHAR(255) NULL	
<code>is_active</code>	TINYINT(1) DEFAULT 1	

4.6 `store_setting` — key-value configuration

Column	Type	Notes
<code>name</code>	VARCHAR(64) PK	e.g. <code>shipping_default_fee</code> , <code>free_shipping_threshold</code> , <code>store_email</code> , <code>store_phone</code> , <code>cod_enabled</code> , <code>maintenance_mode</code> .
<code>value</code>	TEXT	

Keeps one-off knobs out of code and out of dedicated single-row tables. Same idea as PrestaShop's `ps_configuration` / WooCommerce options.

5. Orders & checkout entities PHASE 2

5.1 Changes to existing `t_order` (additive only)

Column	Type	Notes
<code>source</code>	VARCHAR(16) NOT NULL DEFAULT 'manual'	Values: <code>manual</code> , <code>web</code> , <code>intigo</code> . Existing rows become <code>manual</code> automatically; Intigo rows can be backfilled if identifiable. Indexed — the back-office filters on it. (<i>Shopify's <code>source_name</code> , PrestaShop's <code>order origin</code>.</i>)

Nothing else changes. `delivered` and `payment_received` keep their meaning and keep driving every existing report. For web orders they become *derived* flags, synced automatically from the status workflow (§5.3) so all dashboards stay correct without modification.

5.2 `store_order_detail` — web-only order data, one-to-one with `t_order`

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>order_id</code>	INT UNIQUE , FK → <code>t_order.id</code>	<code>ON DELETE CASCADE</code> . Exists only for <code>source='web'</code> orders.
<code>store_user_id</code>	INT NULL, FK → <code>store_user.id</code>	NULL = guest checkout. <code>ON DELETE SET NULL</code> (account deletion must not delete sales records).
<code>status_id</code>	INT, FK → <code>store_order_status.id</code>	Current state; full trail in history table.
<code>payment_method</code>	VARCHAR(24) DEFAULT 'cod'	<code>cod</code> now; <code>card_online</code> when a PSP is added (§7.2).
<code>shipping_fee</code>	DECIMAL(10,3)	Snapshot at checkout (zone fees may change later).
<code>discount_code_id</code>	INT NULL, FK → <code>store_discount_code.id</code>	<code>ON DELETE SET NULL</code> .
<code>discount_amount</code>	DECIMAL(10,3) NULL	Snapshot of the money actually deducted.
<code>customer_email</code>	VARCHAR(180) NULL	For confirmation emails; guests have no account row.
<code>ship_first_name / ship_last_name</code>	VARCHAR(100)	Address snapshot — copied from the chosen/entered address at checkout. The saved address can be edited or deleted later; the order keeps what was shipped. (<code>t_order</code> already snapshots client name/phone — same philosophy.)
<code>ship_phone</code>	VARCHAR(20)	
<code>ship_address_line / ship_city</code>	VARCHAR(255) / VARCHAR(100)	
<code>ship_governorate / ship_postal_code</code>	VARCHAR(64) / VARCHAR(10)	
<code>customer_note</code>	TEXT NULL	"Call before delivery", etc.
<code>tracking_number</code>	VARCHAR(64) NULL	Courier reference (Intigo/Aramex/First Delivery).
<code>created_at / updated_at</code>	DATETIME	

5.3 `store_order_status` + `store_order_status_history`

Statuses live in a **table, not an enum**, with behavior flags — so adding a state ("awaiting exchange") or changing which state triggers an email is data entry, not a deployment.

<code>store_order_status</code>	Type	Notes
<code>id</code>	INT PK AI	

<code>code</code>	VARCHAR(32) UNIQUE	Stable machine name used in code and as frontend translation key.
<code>label_fr</code>	VARCHAR(64)	Shown in emails and back-office (UI also has its own i18n).
<code>color</code>	VARCHAR(7)	Badge color in the back-office list.
<code>position</code>	INT	Display order.
<code>is_final</code>	TINYINT(1)	No transitions allowed out (delivered, cancelled, returned).
<code>restocks</code>	TINYINT(1)	Entering this status returns the items to <code>product.quantity</code> (cancelled, returned).
<code>sets_delivered</code>	TINYINT(1)	Syncs <code>t_order.delivered = 1</code> — keeps every existing report correct.
<code>sets_payment_received</code>	TINYINT(1)	Syncs <code>t_order.payment_received = 1</code> (COD: collected on delivery).
<code>sends_email</code>	TINYINT(1)	Notify the customer on entry (template = <code>code</code>).

Precedent: this is PrestaShop's `ps_order_state` almost verbatim — its rows carry `paid` , `shipped` , `delivery` , `send_email` flags that drive behavior, and `ps_order_history` records every transition. It is the most battle-tested order-state model in e-commerce.

Seed statuses: `pending` (new, stock already taken) → `confirmed` (staff called the customer — the COD norm in Tunisia) → `shipped` (sets_delivered? no — tracking set) → `delivered` (sets_delivered + sets_payment_received for COD) → terminal; plus `cancelled` and `returned` (both `restocks=1`).

store_order_status_history	Type	Notes
<code>id</code>	INT PK AI	
<code>order_detail_id</code>	INT, FK → store_order_detail.id	ON DELETE CASCADE .
<code>status_id</code>	INT, FK → store_order_status.id	
<code>comment</code>	VARCHAR(255) NULL	"Client unreachable, retry tomorrow".
<code>changed_by</code>	VARCHAR(64) NULL	Staff username, <code>customer</code> , or <code>system</code> .
<code>created_at</code>	DATETIME	Answers "when did we ship?" and measures fulfillment speed.

5.4 Checkout flow — how a web order is born

- Checkout calls the **existing order-creation service** (the same code path used by manual orders and Intigo), with `source='web'` . Stock decrements, `sold_units` updates, the client is upserted by phone — all existing, tested logic.
- The same transaction inserts `store_order_detail` (status `pending` , address snapshot, shipping fee from the zone table) and the first history row.
- Stock is taken at order creation — identical to today's behavior, so two simultaneous shoppers cannot buy the last unit. Cancellation restocks via the `restocks` flag.
- The existing 5-second duplicate-order guard protects against double-submit on the web too.
- Order confirmation email is queued (\$6.4).

5.5 `store_delivery_zone`

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>name</code>	VARCHAR(64) UNIQUE	The 24 Tunisian governorates, seeded.
<code>fee</code>	DECIMAL(10,3)	e.g. 7.000 DT Grand Tunis, 9.000 DT elsewhere.
<code>free_above</code>	DECIMAL(10,3) NULL	Free shipping threshold per zone; NULL = use global setting.
<code>estimated_days</code>	VARCHAR(16) NULL	"24-48h" shown at checkout.
<code>is_active</code>	TINYINT(1) DEFAULT 1	Stop delivering somewhere without deleting history.

6. Customer account entities PHASE 3

6.1 `store_user` — shopper login accounts

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>email</code>	VARCHAR(180) UNIQUE	Login identifier (staff log in by username; shoppers by email).
<code>password</code>	VARCHAR(255)	bcrypt via Symfony's hasher. Separate firewall + user provider in <code>security.yaml</code> ; the shop JWT carries <code>ROLE_CUSTOMER</code> only and is useless against staff endpoints.
<code>first_name / last_name</code>	VARCHAR(100)	
<code>phone</code>	VARCHAR(20)	Used to link to <code>client</code> .
<code>client_id</code>	INT NULL UNIQUE , FK → <code>client.id</code>	<code>ON DELETE SET NULL</code> . Linked at registration (phone match) or on first order. Gives the back-office one unified purchase history per human across boutique + web + marketplace.
<code>is_active</code>	TINYINT(1) DEFAULT 1	Soft-disable abusive accounts.
<code>reset_token / reset_token_expires_at</code>	VARCHAR(64) NULL / DATETIME NULL	Password-reset flow (token hashed at rest).
<code>created_at / last_login_at</code>	DATETIME	

Why not put credentials on `client`? 4,000+ `client` rows exist that never asked for an account; mixing auth columns into an operational table couples the inventory app to web security concerns (password column in every staff-visible query, accidental serialization, GDPR-style deletion requests hitting sales history). Sylius learned this the hard way and split `Customer/ShopUser`; we start split.

6.2 `store_address` — saved address book

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>store_user_id</code>	INT, FK → <code>store_user.id</code>	<code>ON DELETE CASCADE</code> — addresses die with the account (orders keep their snapshot).
<code>label</code>	VARCHAR(32) NULL	"Maison", "Bureau".
<code>first_name / last_name / phone</code>	VARCHAR	Recipient may differ from account holder (gifts).
<code>address_line / city / postal_code</code>	VARCHAR	
<code>delivery_zone_id</code>	INT, FK → <code>store_delivery_zone.id</code>	<code>ON DELETE RESTRICT</code> ; drives the fee shown at checkout.
<code>is_default</code>	TINYINT(1) DEFAULT 0	

6.3 `store_cart` / `store_cart_item` + `store_wishlist_item`

Guests keep the current `localStorage` cart — zero backend cost, works today. Logged-in users get a server cart that survives devices; on login the local cart is **merged** into the server cart (quantities added, then `localStorage` cleared). This is the Shopify/PrestaShop behavior users expect.

<code>store_cart</code>	Type	Notes
<code>id</code>	INT PK AI	

<code>store_user_id</code>	INT UNIQUE , FK → <code>store_user.id</code>	One active cart per account. <code>ON DELETE CASCADE</code> .
<code>created_at</code> / <code>updated_at</code>	DATETIME	<code>updated_at</code> enables abandoned-cart reporting later.

store_cart_item	Type	Notes
<code>id</code>	INT PK AI	
<code>cart_id</code>	INT, FK → <code>store_cart.id</code>	<code>ON DELETE CASCADE</code> ; <code>UNIQUE(cart_id , product_id)</code> .
<code>product_id</code>	INT, FK → <code>product.id</code>	<code>ON DELETE CASCADE</code> .
<code>quantity</code>	INT	No price column — carts always display the live price; the only price snapshot happens at order creation. Stale cart prices are a classic source of disputes.

store_wishlist_item	Type	Notes
<code>id</code>	INT PK AI	
<code>store_user_id</code>	INT, FK → <code>store_user.id</code>	<code>ON DELETE CASCADE</code> ; <code>UNIQUE(store_user_id , product_id)</code> .
<code>product_id</code>	INT, FK → <code>product.id</code>	<code>ON DELETE CASCADE</code> .
<code>created_at</code>	DATETIME	The existing <code>wishlist.service.ts</code> keeps localStorage for guests; syncs on login like the cart.

6.4 Mail: `store_email_log` + `store_contact_message`

Email **templates are Twig files** in the repo (versioned, reviewable — like the existing invoice template), not DB rows. What goes in the DB is the **log**: shared-hosting SMTP fails silently often enough that "did the customer get the confirmation?" must be answerable from the back-office. `symfony/mailer` is already installed.

store_email_log	Type	Notes
<code>id</code>	INT PK AI	
<code>recipient</code> / <code>subject</code>	VARCHAR	
<code>template</code>	VARCHAR(64)	<code>order_confirmation</code> , <code>order_shipped</code> , <code>password_reset</code> , <code>welcome</code> , <code>contact_copy</code> .
<code>order_id</code>	INT NULL, FK → <code>t_order.id</code>	Jump from an order to its emails. <code>ON DELETE SET NULL</code> .
<code>status</code>	VARCHAR(12)	<code>sent</code> / <code>failed</code> (with <code>error</code> TEXT NULL).
<code>created_at</code>	DATETIME	

store_contact_message	Type	Notes
<code>id</code>	INT PK AI	Backs the storefront's existing contact page.
<code>name</code> / <code>email</code> / <code>phone</code>	VARCHAR (phone NULL)	
<code>subject</code> / <code>message</code>	VARCHAR / TEXT	
<code>is_read</code>	TINYINT(1) DEFAULT 0	Inbox-style screen in the back-office module.
<code>created_at</code>	DATETIME	

7. Growth entities PHASE 4 (designed now, built when needed)

7.1 `store_discount_code`

Column	Type	Notes
<code>id</code> / <code>code</code>	INT PK / VARCHAR(32) UNIQUE	Case-insensitive lookup.
<code>type</code>	VARCHAR(8)	<code>percent</code> <code>fixed</code> — mirrors the two discount modes the PDF invoice already supports.
<code>value</code>	DECIMAL(10,3)	
<code>min_order_amount</code>	DECIMAL(10,3) NULL	
<code>starts_at</code> / <code>ends_at</code>	DATETIME NULL	
<code>max_uses</code> / <code>times_used</code> / <code>max_uses_per_user</code>	INT NULL / INT / INT NULL	Per-user limit checked against <code>store_order_detail.discount_code_id</code> .
<code>is_active</code>	TINYINT(1) DEFAULT 1	

7.2 `store_payment_transaction`

For when online card payment arrives (Tunisian PSPs: Paymee, Konnect, ClicToPay). COD needs no row here — `t_order.payment_received` already covers it.

Column	Type	Notes
<code>id</code>	INT PK AI	
<code>order_id</code>	INT, FK → <code>t_order.id</code>	One order may have several attempts (failed → retry). <code>ON DELETE RESTRICT</code> — never lose money trails.
<code>provider</code> / <code>provider_ref</code>	VARCHAR(24) / VARCHAR(128)	PSP name + their transaction id.
<code>amount</code>	DECIMAL(10,3)	
<code>status</code>	VARCHAR(12)	<code>initiated</code> / <code>success</code> / <code>failed</code> / <code>refunded</code> .
<code>payload</code>	JSON NULL	Raw PSP callback, for dispute forensics.
<code>created_at</code> / <code>updated_at</code>	DATETIME	

7.3 Smaller tables

Table	Columns	Purpose
<code>store_product_review</code>	<code>id</code> , <code>product_id</code> FK, <code>store_user_id</code> FK NULL, <code>rating</code> TINYINT, <code>title</code> , <code>body</code> , <code>is_approved</code> , <code>created_at</code>	Reviews with back-office moderation (<code>is_approved</code> default 0).
<code>store_banner</code>	<code>id</code> , <code>image</code> , <code>link</code> , <code>title</code> NULL, <code>position</code> , <code>is_active</code> , <code>starts_at</code> / <code>ends_at</code> NULL	Homepage slider/banners managed from the back-office instead of redeploys.
<code>store_newsletter_subscription</code>	<code>id</code> , <code>email</code> UNIQUE, <code>is_active</code> , <code>created_at</code> , <code>unsubscribed_at</code> NULL	Footer signup form.

8. Existing entities — reuse map

Existing table	Role in the e-commerce app	Changes
----------------	----------------------------	---------

<code>product</code>	Single source of truth for stock (<code>quantity</code>), base price (<code>selling_price</code>), cost (<code>buying_price</code> , never exposed publicly), code, sold units. Web visibility = has a published <code>store_product_detail</code> .	None.
<code>t_order</code>	Every sale from every channel. Web checkout calls the existing creation service → stock, KPIs, charts, Excel exports and invoices all keep working.	Add <code>source</code> VARCHAR(16) DEFAULT 'manual'. <code>delivered</code> / <code>payment_received</code> become synced flags for web orders.
<code>order_to_product</code>	Order lines for web orders too — quantity + selling price per line, exactly as today.	None.
<code>client</code>	The customer profile across all channels, still keyed by phone, still auto-upserted at order creation. Web guest checkout uses the same upsert. <code>store_user</code> links to it for logged-in shoppers.	None. (Its free-text <code>address</code> column stays for inventory use; structured addresses live in <code>store_address</code> / order snapshots.)
<code>user / role</code>	Staff only, exactly as today. Back-office e-commerce screens are gated by the existing roles (e.g. content editing for MANAGER+).	None.
<code>product_history</code> , <code>broken_product</code> , <code>inventory_snapshot</code> , <code>document</code> , <code>folder</code>	Untouched. Restocks and damage flow into stock as today; snapshots/reports automatically include web revenue because it lives in <code>t_order</code> .	None.

9. Deliberate simplifications (and their escape hatches)

Not building	Why	If it's ever needed
Product variants engine (size/shade matrices)	Each variant already exists as its own inventory product with its own barcode — that's how the warehouse scans and counts. A variants layer would duplicate stock-keeping.	Add <code>variant_group</code> VARCHAR NULL on <code>store_product_detail</code> ; storefront groups rows sharing it into one page with a selector. No migration of existing data needed.
EAV / dynamic attributes (Magento-style)	Killed many shops' performance for flexibility a 300-product beauty store never uses.	A nullable JSON <code>attributes</code> column on <code>store_product_detail</code> covers "Contentance: 50ml" display tables.
Multi-currency / multi-language content	TND only; content in French (UI chrome is already translatable in the Angular apps).	Add <code>store_product_detail_translation</code> later; slugs already unique per row.
Reserved-stock / inventory holds	Stock decrements at order creation (existing behavior), so nothing needs reserving.	Only relevant if checkout ever stops creating orders immediately (e.g. pay-first flows) — then add a <code>reserved_until</code> on cart items.
Server-side guest carts	localStorage works today, zero backend load, no session GC.	<code>store_cart.store_user_id</code> is already nullable-ready; add a <code>token</code> column and a cookie.

10. Decisions to confirm before Phase 1 starts

- **Image storage:** proposed = backend disk under `public/uploads/products/` (same pattern as Documents), resized at upload. Confirm hosting disk space is adequate for ~300 products × 2–3 images.
- **Brand entity in Phase 1?** Cheap now; confirm the client wants brand filtering/pages.
- **Who curates content?** Publishing 300 products (photos, descriptions) is the real project bottleneck — decide who does data entry and whether to import anything from the old PrestaShop database (`fpmiqfdv_inventory.sql` sits in the workspace; the old shop's `ps_product_lang` descriptions and images may be importable, saving weeks).
- **Order numbering:** web orders share `t_order.id` sequence with manual orders. If the client wants "WEB-1024"-style references, add a display-reference column on `store_order_detail` — decide before first real order.
- **Prod PHP upgrade** (7.3 → 8.1/8.2) before the public API goes live — already discussed; affects dependency security patching, not this schema.

11. Suggested build order

#	Scope	Tables	Outcome
1	PHASE 1 Catalog schema + Doctrine migrations + public read API (/api/shop/* : category tree, category products, product by slug, search) + back-office "Boutique" module (categories, product publishing, images)	store_category , store_product_detail , store_product_to_category , store_product_image , store_brand , store_setting	Storefront drops catalog-data.ts ; the site shows real, staff-managed products.
2	PHASE 2 Guest checkout (COD), order statuses + history, delivery zones, transactional email, contact form	t_order.source , store_order_detail , store_order_status(+history) , store_delivery_zone , store_email_log , store_contact_message	The site sells. Staff manage web orders inside the inventory app.
3	PHASE 3 Accounts: registration/login (separate JWT firewall), address book, order history page, wishlist + cart sync, password reset. Unlocks the template's auth-gated pages.	store_user , store_address , store_cart(+items) , store_wishlist_item	Full customer experience; client↔account linking gives unified history.
4	PHASE 4 Promo codes, online payment, reviews, banners, newsletter	store_discount_code , store_payment_transaction , store_product_review , store_banner , store_newsletter_subscription	Marketing & growth tooling.